

test any of this code is in the embedded target. Virtually every bit of this code knows it is in a special microprocessor architecture, using “extended” C constructs³ that tie the code to a particular tool chain and microprocessor. There is no way for this code to have a long useful life unless the product never needs to be moved to a different hardware environment.

This application works: The engineer passed the App-titude test. But the application can’t be said to have a clean embedded architecture.

THE TARGET-HARDWARE BOTTLENECK

There are many special concerns that embedded developers have to deal with that non-embedded developers do not—for example, limited memory space, real-time constraints and deadlines, limited IO, unconventional user interfaces, and sensors and connections to the real world. Most of the time the hardware is concurrently developed with the software and firmware. As an engineer developing code for this kind of system, you may have no place to run the code. If that’s not bad enough, once you get the hardware, it is likely that the hardware will have its own defects, making software development progress even slower than usual.

Yes, embedded is special. Embedded engineers are special. But embedded development is not *so* special that the principles in this book are not applicable to embedded systems.

One of the special embedded problems is *the target-hardware bottleneck*. When embedded code is structured without applying clean architecture principles and practices, you will often face the scenario in which you can test your code only on the target. If the target is the only place where testing is possible, the target-hardware bottleneck will slow you down.

A CLEAN EMBEDDED ARCHITECTURE IS A TESTABLE EMBEDDED ARCHITECTURE

Let’s see how to apply some of the architectural principles to embedded software and firmware to help you eliminate the target-hardware bottleneck.

Layers

Layering comes in many flavors. Let’s start with three layers, as shown in [Figure](#)

[29.1](#). At the bottom, there is the hardware. As Doug warns us, due to technology advances and Moore's law, the hardware will change. Parts become obsolete, and new parts use less power or provide better performance or are cheaper. Whatever the reason, as an embedded engineer, I don't want to have a bigger job than is necessary when the inevitable hardware change finally happens.

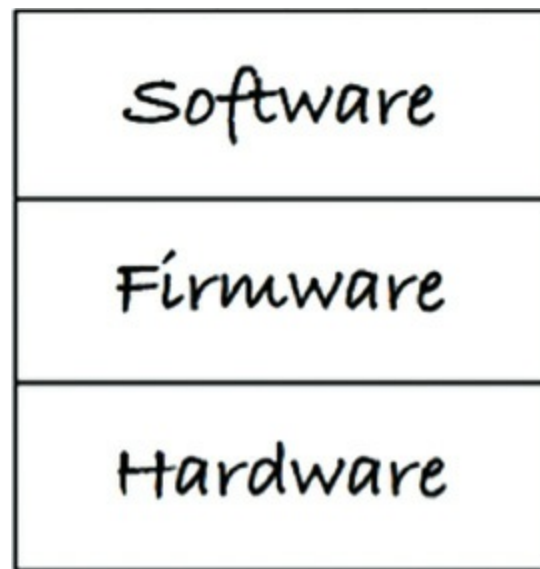


Figure 29.1 Three layers

The separation between hardware and the rest of the system is a given—at least once the hardware is defined ([Figure 29.2](#)). Here is where the problems often begin when you are trying to pass the App-titude test. There is nothing that keeps hardware knowledge from polluting all the code. If you are not careful about where you put things and what one module is allowed to know about another module, the code will be very hard to change. I'm not just talking about when the hardware changes, but when the user asks for a change, or when a bug needs to be fixed.

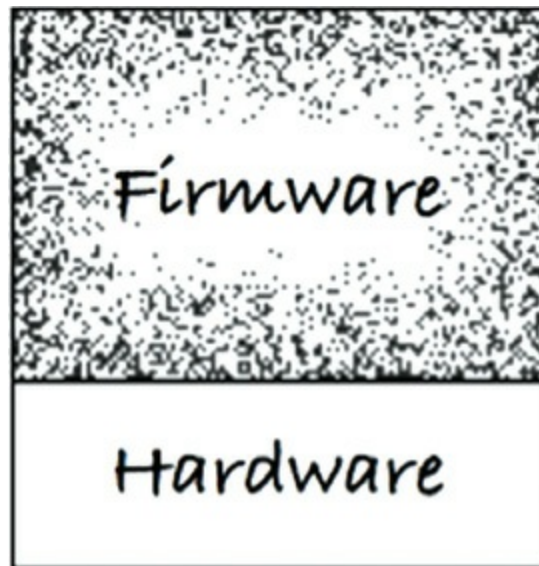


Figure 29.2 Hardware must be separated from the rest of the system

Software and firmware intermingling is an anti-pattern. Code exhibiting this anti-pattern will resist changes. In addition, changes will be dangerous, often leading to unintended consequences. Full regression tests of the whole system will be needed for minor changes. If you have not created externally instrumented tests, expect to get bored with manual tests—and then you can expect new bug reports.

The Hardware Is a Detail

The line between software and firmware is typically not so well defined as the line between code and hardware, as shown in [Figure 29.3](#).

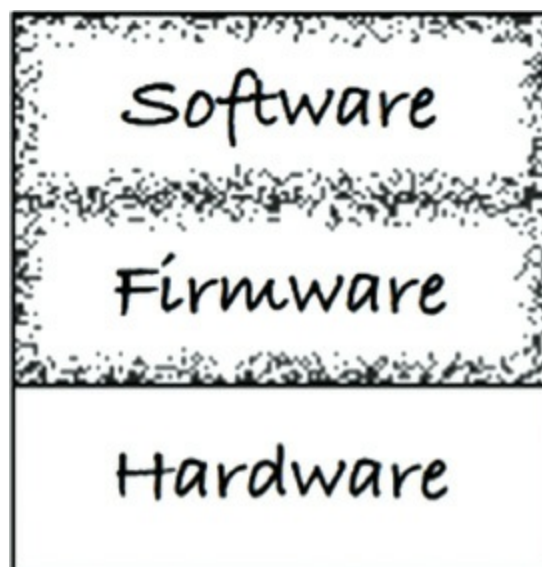


Figure 29.3 The line between software and firmware is a bit fuzzier than the line between code and

hardware

One of your jobs as an embedded software developer is to firm up that line. The name of the boundary between the software and the firmware is the hardware abstraction layer (HAL) ([Figure 29.4](#)). This is not a new idea: It has been in PCs since the days before Windows.

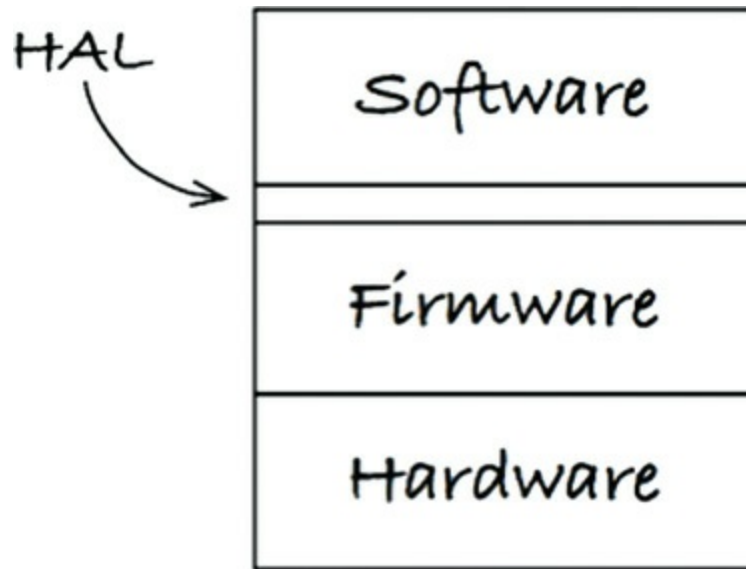


Figure 29.4 The hardware abstraction layer

The HAL exists for the software that sits on top of it, and its API should be tailored to that software's needs. As an example, the firmware can store bytes and arrays of bytes into flash memory. In contrast, the application needs to store and read name/value pairs to some persistence mechanism. The software should not be concerned that the name/value pairs are stored in flash memory, a spinning disk, the cloud, or core memory. The HAL provides a service, and it does not reveal to the software how it does it. The flash implementation is a detail that should be hidden from software.

As another example, an LED is tied to a GPIO bit. The firmware could provide access to the GPIO bits, where a HAL might provide `Led_TurnOn(5)`. That is a pretty low-level hardware abstraction layer. Let's consider raising the level of abstraction from a hardware perspective to the software/product perspective. What is the LED indicating? Suppose that it indicated low battery power. At some level, the firmware (or a board support package) could provide `Led_TurnOn(5)`, while the HAL provides `Indicate_LowBattery()`. You can see the HAL expressing services needed by the application. You can also see that layers may contain layers. It is more of a repeating fractal pattern than a limited set of predefined layers. The GPIO

assignments are details that should be hidden from the software.

DON'T REVEAL HARDWARE DETAILS TO THE USER OF THE HAL

A clean embedded architecture's software is testable *off* the target hardware. A successful HAL provides that seam or set of substitution points that facilitate off-target testing.

The Processor Is a Detail

When your embedded application uses a specialized tool chain, it will often provide header files to *help you*.⁴ These compilers often take liberties with the C language, adding new keywords to access their processor features. The code will look like C, but it is no longer C.

Sometimes vendor-supplied C compilers provide what look like global variables to give access directly to processor registers, IO ports, clock timers, IO bits, interrupt controllers, and other processor functions. It is helpful to get access to these things easily, but realize that any of your code that uses these helpful facilities is no longer C. It won't compile for another processor, or maybe even with a different compiler for the same processor.

I would hate to think that the silicon and tool provider is being cynical, tying your product to the compiler. Let's give the provider the benefit of a doubt by assuming that it is truly trying to help. But now it's up to you to use that help in a way that does not hurt in the future. You will have to limit which files are allowed to know about the C extensions.

Let's look at this header file designed for the ACME family of DSPs—you know, the ones used by Wile E. Coyote:

[Click here to view code image](#)

```
#ifndef _ACME_STD_TYPES
#define _ACME_STD_TYPES
```

[Click here to view code image](#)

```

#if defined(_ACME_X42)
    typedef unsigned int      Uint_32;
    typedef unsigned short    Uint_16;
    typedef unsigned char     Uint_8;

    typedef int               Int_32;
    typedef short             Int_16;
    typedef char              Int_8;

#elif defined(_ACME_A42)
    typedef unsigned long     Uint_32;
    typedef unsigned int      Uint_16;
    typedef unsigned char     Uint_8;

    typedef long              Int_32;
    typedef int               Int_16;
    typedef char              Int_8;
#else
    #error <acmetypes.h> is not supported for this environment
#endif

#endif

```

The `acmetypes.h` header file should not be used directly. If you do, your code gets tied to one of the ACME DSPs. You are using an ACME DSP, you say, so what is the harm? You can't compile your code unless you include this header. If you use the header and define `_ACME_X42` or `_ACME_A42`, your integers will be the wrong size if you try to test your code off-target. If that is not bad enough, one day you'll want to port your application to another processor, and you will have made that task much more difficult by not choosing portability and by not limiting what files know about ACME.

Instead of using `acmetypes.h`, you should try to follow a more standardized path and use `stdint.h`. But what if the target compiler does not provide `stdint.h`? You can write this header file. The `stdint.h` you write for target builds uses the `acmetypes.h` for target compiles like this:

[Click here to view code image](#)

```

#ifndef _STDINT_H_
#define _STDINT_H_

#include <acmetypes.h>

```

```

typedef Uint_32 uint32_t;
typedef Uint_16 uint16_t;
typedef Uint_8  uint8_t;

typedef Int_32  int32_t;
typedef Int_16  int16_t;
typedef Int_8   int8_t;

#endif

```

Having your embedded software and firmware use `stdint.h` helps keep your code clean and portable. Certainly, all of the *software* should be processor independent, but not all of the *firmware* can be. This next code snippet takes advantage of special extensions to C that gives your code access to the peripherals in the micro-controller. It's likely your product uses this micro-controller so that you can use its integrated peripherals. This function outputs a line that says "hi" to the serial output port. (This example is based on real code from the wild.)

[Click here to view code image](#)

```

void say_hi()
{
    IE = 0b11000000;
    SBUF0 = (0x68);
    while(TI_0 == 0);
    TI_0 = 0;
    SBUF0 = (0x69);
    while(TI_0 == 0);
    TI_0 = 0;
    SBUF0 = (0x0a);
    while(TI_0 == 0);
    TI_0 = 0;
    SBUF0 = (0x0d);
    while(TI_0 == 0);
    TI_0 = 0;
    IE = 0b11010000;
}

```

There are lots of problems with this small function. One thing that might jump out at you is the presence of `0b11000000`. This binary notation is cool; can C do that? Unfortunately, no. A few other problems relate to this code directly using the custom C extensions:

IE: Interrupt enable bits.

SBUF0: Serial output buffer.

TI_0: Serial transmit buffer empty interrupt. Reading a 1 indicates the buffer is empty.

The uppercase variables actually access micro-controller built-in peripherals. If you want to control interrupts and output characters, you must use these peripherals. Yes, this is convenient—but it's not C.

A clean embedded architecture would use these device access registers directly in very few places and confine them totally to the *firmware*. Anything that knows about these registers becomes *firmware* and is consequently bound to the silicon. Tying code to the processor will hurt you when you want to get code working before you have stable hardware. It will also hurt you when you move your embedded application to a new processor.

If you use a micro-controller like this, your firmware could isolate these low-level functions with some form of a *processor abstraction layer* (PAL). Firmware above the PAL could be tested off-target, making it a little less firm.

The Operating System Is a Detail

A HAL is necessary, but is it sufficient? In bare-metal embedded systems, a HAL may be all you need to keep your code from getting too addicted to the operating environment. But what about embedded systems that use a real-time operating system (RTOS) or some embedded version of Linux or Windows?

To give your embedded code a good chance at a long life, you have to treat the operating system as a detail and protect against OS dependencies.

The software accesses the services of the operating environment through the OS. The OS is a layer separating the software from firmware ([Figure 29.5](#)). Using an OS directly can cause problems. For example, what if your RTOS supplier is bought by another company and the royalties go up, or the quality goes down? What if your needs change and your RTOS does not have the capabilities that you now require? You'll have to change lots of code. These won't just be simple syntactical changes due to the new OS's API, but will likely have to adapt semantically to the new OS's different capabilities and primitives.



Figure 29.5 Adding in an operating system

A clean embedded architecture isolates software from the operating system, through an *operating system abstraction layer* (OSAL) ([Figure 29.6](#)). In some cases, implementing this layer might be as simple as changing the name of a function. In other cases, it might involve wrapping several functions together.

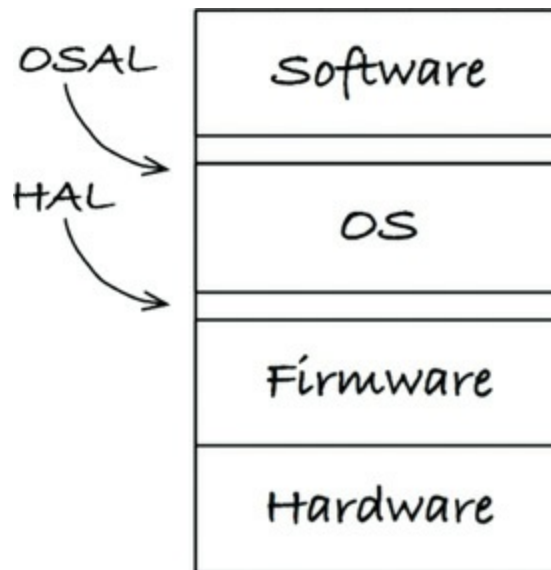


Figure 29.6 The operating system abstraction layer

If you have ever moved your software from one RTOS to another, you know it is painful. If your software depended on an OSAL instead of the OS directly, you would largely be writing a new OSAL that is compatible with the old OSAL. Which would you rather do: modify a bunch of complex existing code, or write new code to a defined interface and behavior? This is not a trick question. I choose the latter.

You might start worrying about code bloat about now. Really, though, the layer becomes the place where much of the duplication around using an OS is isolated. This duplication does not have to impose a big overhead. If you define an OSAL, you can also encourage your applications to have a common structure. You might provide message passing mechanisms, rather than having every thread handcraft its concurrency model.

The OSAL can help provide test points so that the valuable application code in the software layer can be tested off-target and off-OS. A clean embedded architecture's software is testable off the target operating system. A successful OSAL provides that seam or set of substitution points that facilitate off-target testing.

PROGRAMMING TO INTERFACES AND SUBSTITUTABILITY

In addition to adding a HAL and potentially an OSAL inside each of the major layers (software, OS, firmware, and hardware), you can—and should—apply the principles described throughout this book. These principles encourage separation of concerns, programming to interfaces, and substitutability.

The idea of a layered architecture is built on the idea of programming to interfaces. When one module interacts with another through an interface, you can substitute one service provider for another. Many readers will have written their own small version of `printf` for deployment in the target. As long as the interface to your `printf` is the same as the standard version of `printf`, you can override the service one for the other.

One basic rule of thumb is to use header files as interface definitions. When you do so, however, you have to be careful about what goes in the header file. Limit header file contents to function declarations as well as the constants and struct names that are needed by the function.

Don't clutter the interface header files with data structures, constants, and typedefs that are needed by only the implementation. It's not just a matter of clutter: That clutter will lead to unwanted dependencies. Limit the visibility of the implementation details. Expect the implementation details to change. The fewer places where code knows the details, the fewer places where code will have to be tracked down and modified.

A clean embedded architecture is testable within the layers because modules interact

through interfaces. Each interface provides that seam or substitution point that facilitates off-target testing.

DRY CONDITIONAL COMPILATION DIRECTIVES

One use of substitutability that is often overlooked relates to how embedded C and C++ programs handle different targets or operating systems. There is a tendency to use conditional compilation to turn on and off segments of code. I recall one especially problematic case where the statement `#ifdef BOARD_V2` was mentioned several thousand times in a telecom application.

This repetition of code violates the Don't Repeat Yourself (DRY) principle.⁵ If I see `#ifdef BOARD_V2` once, it's not really a problem. *Six thousand times* is an extreme problem. Conditional compilation identifying the target-hardware's type is often repeated in embedded systems. But what else can we do?

What if there is a hardware abstraction layer? The hardware type would become a detail hidden under the HAL. If the HAL provides a set of interfaces, instead of using conditional compilation, we could use the linker or some form of runtime binding to connect the software to the hardware.

CONCLUSION

People who are developing embedded software have a lot to learn from experiences outside of embedded software. If you are an embedded developer who has picked up this book, you will find a wealth of software development wisdom in the words and ideas.

Letting all code become firmware is not good for your product's long-term health. Being able to test only in the target hardware is not good for your product's long-term health. A clean embedded architecture is good for your product's long-term health.

1. https://insights.sei.cmu.edu/sei_blog/2011/08/the-growing-importance-of-sustaining-software-for-the-dod.html
2. The hardware device that controls the serial port.
3. Some silicon providers add keywords to the C language to make accessing the registers and IO ports simple from C. Unfortunately, once that is done, the code is no longer C.
4. This statement intentionally uses HTML.

[5](#). Hunt and Thomas, The *Pragmatic Programmer*.

VI

DETAILS

THE DATABASE IS A DETAIL



From an architectural point of view, the database is a non-entity—it is a detail that does not rise to the level of an architectural element. Its relationship to the architecture of a software system is rather like the relationship of a doorknob to the architecture of your home.

I realize that these are fighting words. Believe me, I've had the fight. So let me be clear: I am not talking about the data model. The structure you give to the data within your application is highly significant to the architecture of your system. But the database is not the data model. The database is piece of software. The database is a utility that provides access to the data. From the architecture's point of view, that utility is irrelevant because it's a low-level detail—a mechanism. And a good architect does not allow low-level mechanisms to pollute the system architecture.

RELATIONAL DATABASES

Edgar Codd defined the principles of relational databases in 1970. By the mid-1980s, the relational model had grown to become the dominant form of data storage. There was a good reason for this popularity: The relational model is elegant, disciplined, and robust. It is an excellent data storage and access technology.

But no matter how brilliant, useful, and mathematically sound a technology it is, it is still just a technology. And that means it's a detail.

While relational tables may be convenient for certain forms of data access, there is nothing architecturally significant about arranging data into rows within tables. The use cases of your application should neither know nor care about such matters. Indeed, knowledge of the tabular structure of the data should be restricted to the lowest-level utility functions in the outer circles of the architecture.

Many data access frameworks allow database rows and tables to be passed around the system as objects. Allowing this is an architectural error. It couples the use cases, business rules, and in some cases even the UI to the relational structure of the data.

WHY ARE DATABASE SYSTEMS SO PREVALENT?

Why are software systems and software enterprises dominated by database systems? What accounts for the preeminence of Oracle, MySQL, and SQL Server? In a word: disks.

The rotating magnetic disk was the mainstay of data storage for five decades. Several generations of programmers have known no other form of data storage. Disk technology has grown from huge stacks of massive platters 48 inches in diameter that weighed thousands of pounds and held 20 megabytes, to single thin circles, 3 inches in diameter, that weigh just a few grams and hold a terabyte or more. *It's been a wild ride.* And throughout that ride programmers have been plagued by one fatal trait of disk technology: Disks are *slow*.

On a disk, data is stored within circular tracks. Those tracks are divided into sectors that hold a convenient number of bytes, often 4K. Each platter may have hundreds of tracks, and there can be a dozen or so platters. If you want to read a particular byte

off the disk, you have to move the head to the proper track, wait for the disk to rotate to the proper sector, read all 4K of that sector into RAM, and then index into that RAM buffer to get the byte you want. And all that takes time—milliseconds of times.

Milliseconds might not seem like a lot, but a millisecond is a million times longer than the cycle time of most processors. If that data was not on a disk, it could be accessed in nanoseconds, instead of milliseconds.

To mitigate the time delay imposed by disks, you need indexes, caches, and optimized query schemes; and you need some kind of regular means of representing the data so that these indexes, caches, and query schemes know what they are working with. In short, you need a data access and management system. Over the years these systems have split into two distinct kinds: file systems and relational database management systems (RDBMS).

File systems are document based. They provide a natural and convenient way to store whole documents. They work well when you need to save and retrieve a set of documents by name, but they don't offer a lot of help when you're searching the content of those documents. It's easy to find a file named `login.c`, but it's hard, and slow, to find every `.c` file that has a variable named `x` in it.

Database systems are content based. They provide a natural and convenient way to find records based on their content. They are very good at associating multiple records based on some bit of content that they all share. Unfortunately, they are rather poor at storing and retrieving opaque documents.

Both of these systems organize the data on disk so that it can be stored and retrieved in as efficient a way as possible, given their particular access needs. Each has their own scheme for indexing and arranging the data. In addition, each eventually brings the relevant data into RAM, where it can be quickly manipulated.

WHAT IF THERE WERE NO DISK?

As prevalent as disks once were, they are now a dying breed. Soon they will have gone the way of tape drives, floppy drives, and CDs. They are being replaced by RAM.

Ask yourself this question: When all the disks are gone, and all your data is stored in